Geth Init 명령어 상세 분석

`geth init <genesis.json>` 명령어의 내부 동작 과정을 단계별로 분석한 문서입니다.

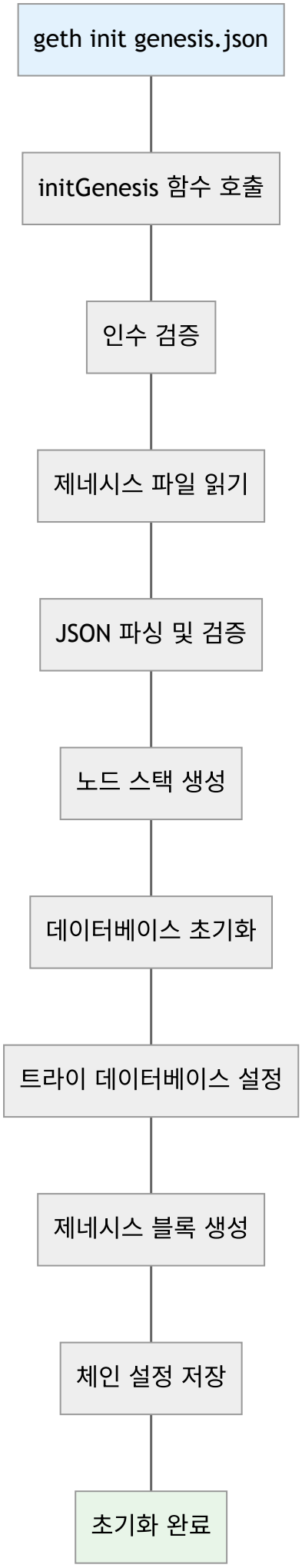
> **분석 대상:** `geth init genesis.json`

> **진입점:** `initGenesis()` 함수

> **소스 파일:** `cmd/geth/chaincmd.go`

> **목적:** 새로운 블록체인 네트워크의 제네시스 블록 초기화

🚀 전체 실행 흐름 개요


geth init genesis.json
initGenesis 함수 호출
인수 검증
제네시스 파일 읽기
JSON 파싱 및 검증
노드 스택 생성
데이터베이스 초기화
트라이 데이터베이스 설정
제네시스 블록 생성
체인 설정 저장
초기화 완료

📋 단계별 상세 분석

1단계: 명령어 정의 및 플래그 설정

```
var  initCommand = &cli.Command{
Action: initGenesis, // 실행할 함수
Name: "init", // 명령어 이름
Usage: "Bootstrap and initialize a new genesis block",
ArgsUsage: "<genesisPath>", // 필수 인수: 제네시스 파일 경로
Flags: slices.Concat([]cli.Flag{
utils.CachePreimagesFlag, // 프리이미지 캐시 활성화
utils.OverrideOsaka, // 오사카 하드포크 오버라이드
utils.OverrideVerkle, // 버클 트리 오버라이드
}, utils.DatabaseFlags), // 데이터베이스 관련 플래그들
Description: `
The init command initializes a new genesis block and definition for the network.
This is a destructive action and changes the network in which you will be
participating.

It expects the genesis file as argument.`,
}
```

지원하는 주요 플래그:

- `--cache.preimages` : 해시 프리이미지 캐시 활성화
- `--override.osaka` : 오사카 하드포크 블록 번호 오버라이드
- `--override.verkle` : 버클 트리 하드포크 블록 번호 오버라이드
- `--datadir` : 데이터 디렉토리 경로
- `--db.engine` : 데이터베이스 엔진 (leveldb/pebble)

2단계: 진입점 함수 (`initGenesis()`)

```
// initGenesis는 주어진 JSON 형식의 제네시스 파일을 초기화하고
// 이를 0번 블록(즉, 제네시스)으로 작성하거나 실패 시 하드 실패합니다.
func initGenesis(ctx *cli.Context) error {
// 1. 인수 개수 검증
if ctx.Args().Len() != 1 {
utils.Fatalf("need genesis.json file as the only argument")
}
// 2. 제네시스 파일 경로 검증
genesisPath := ctx.Args().First()
if len(genesisPath) == 0 {
utils.Fatalf("invalid path to genesis file")
}
// 3. 제네시스 파일 열기
file, err := os.Open(genesisPath)
if err != nil {
utils.Fatalf("Failed to read genesis file: %v", err)
```

```
}
defer file.Close()


// 4. JSON 디코딩
genesis := new(core.Genesis)
if err := json.NewDecoder(file).Decode(genesis); err != nil {
utils.Fatalf("invalid genesis file: %v", err)
}
// ... 계속
}
```

동작 상세:

1. 인수 검증:

- 정확히 1개의 인수(제네시스 파일 경로)만 허용
- 빈 경로나 잘못된 경로 시 즉시 종료

2. 파일 시스템 접근:

- `os.Open()` 으로 제네시스 파일 읽기 전용 열기
- `defer file.Close()` 로 안전한 파일 핸들 정리

3. JSON 파싱:

- `core.Genesis` 구조체로 디코딩
- 잘못된 JSON 형식 시 상세 오류 메시지와 함께 종료

3단계: 노드 스택 및 데이터베이스 초기화

```
// 5. 풀 노드와 라이트 노드 데이터베이스 모두 열기 및 초기화
stack, _ := makeConfigNode(ctx)
defer stack.Close()

// 6. 하드포크 오버라이드 설정
var overrides core.ChainOverrides
if ctx.IsSet(utils.OverrideOsaka.Name) {
v := ctx.Uint64(utils.OverrideOsaka.Name)
overrides.OverrideOsaka = &v
}
if ctx.IsSet(utils.OverrideVerkle.Name) {
v := ctx.Uint64(utils.OverrideVerkle.Name)
overrides.OverrideVerkle = &v
}

// 7. 체인 데이터베이스 생성
chaindb := utils.MakeChainDatabase(ctx, stack, false)
defer chaindb.Close()

// 8. 트라이 데이터베이스 생성
triedb := utils.MakeTrieDatabase(ctx, chaindb,
ctx.Bool(utils.CachePreimagesFlag.Name), // 프리이미지 캐시
```

```
false, // 읽기 전용 아님
genesis.IsVerkle()) // 버클 트리 여부
defer triedb.Close()
```

makeConfigNode 내부 동작:

```
func makeConfigNode(ctx *cli.Context) (*node.Node, gethConfig) {
// 1. 기본 설정 로드
cfg := loadBaseConfig(ctx)
// 2. 노드 스택 생성
stack, err := node.New(&cfg.Node)
if err != nil {
utils.Fatalf("Failed to create the protocol stack: %v", err)
}
// 3. 계정 매니저 백엔드 설정
if err := setAccountManagerBackends(stack.Config(),
stack.AccountManager(),
stack.KeyStoreDir()); err != nil {
utils.Fatalf("Failed to set account manager backends: %v", err)
}

return stack, cfg
}
```

데이터베이스 초기화 과정:

1. 체인 데이터베이스 (`chaindb`)

- 블록, 트랜잭션, 영수증 저장
- LevelDB 또는 Pebble 엔진 사용
- 기본 위치: `<datadir>/geth/chaindata/`

2. 트라이 데이터베이스 (`triedb`)

- 상태 트라이 (계정, 스토리지) 관리
- 프리이미지 캐시 옵션 지원
- 버클 트리 지원 (실험적)

4단계: 제네시스 블록 생성 및 저장

```
// 9. 제네시스 블록 설정 및 생성
_, hash, compatErr, err := core.SetupGenesisBlockWithOverride(
chaindb, // 체인 데이터베이스
triedb, // 트라이 데이터베이스
genesis, // 제네시스 설정
&overrides) // 하드포크 오버라이드

// 10. 오류 처리
if err != nil {
utils.Fatalf("Failed to write genesis block: %v", err)
}
if compatErr != nil {
```

```
utils.Fatalf("Failed to write chain config: %v", compatErr)
}

// 11. 성공 로그
log.Info("Successfully wrote genesis state",
"database", "chaindata",
"hash", hash)

return nil
```

SetupGenesisBlockWithOverride 내부 동작:

```
// core/genesis.go의 핵심 함수
func SetupGenesisBlockWithOverride(chaindb ethdb.Database,
triedb *trie.Database,
genesis *Genesis,
overrides *ChainOverrides) (common.Hash, common.Hash, *params.ChainConfig, error
// 1. 기존 제네시스 블록 확인
stored := rawdb.ReadCanonicalHash(chaindb, 0)
if stored != (common.Hash{}) {
// 기존 제네시스가 있으면 호환성 검사
return checkGenesisCompatibility(chaindb, genesis)
}
```

```
// 2. 새 제네시스 블록 생성
if  genesis == nil {
log.Info("Writing default main-net genesis block")
genesis = DefaultGenesisBlock()
}
// 3. 제네시스 상태 커밋
block, err := genesis.Commit(chaindb, triedb)
if  err != nil {
return common.Hash{}, common.Hash{}, nil, err
}
// 4. 체인 설정 저장
rawdb.WriteChainConfig(chaindb, block.Hash(), genesis.Config)
return  block.Hash(), block.Root(), genesis.Config, nil
}
```

5단계: 제네시스 상태 생성 과정

제네시스 파일 구조 예시:

```
{
"config":  {
"chainId":  1337,
"homesteadBlock":  0,
"eip150Block":  0,
```

```
"eip155Block": 0,
"eip158Block": 0,
"byzantiumBlock": 0,
"constantinopleBlock": 0,
"petersburgBlock": 0,
"istanbulBlock": 0,
"berlinBlock": 0,
"londonBlock": 0
},
"difficulty": "0x1",
"gasLimit": "0x8000000",
"alloc": {
"0x742d35Cc6634C0532925a3b8D4C9db96c4b4d8b": {
"balance": "0x10000000000000000000"
}
}
}
```

상태 생성 과정:

```
// genesis.Commit() 내부
func (g *Genesis) Commit(db ethdb.Database, triedb *trie.Database) (*types.Block
// 1. 상태 데이터베이스 생성
statedb, err := state.New(common.Hash{}, state.NewDatabase(triedb, nil))
```

```
if err != nil {
return nil, err
}
// 2. 제네시스 계정들 생성
for addr, account := range g.Alloc {
statedb.AddBalance(addr, account.Balance)
statedb.SetCode(addr, account.Code)
statedb.SetNonce(addr, account.Nonce)
for key, value := range account.Storage {
statedb.SetState(addr, key, value)
}
}
// 3. 상태 루트 계산 및 커밋
root, err := statedb.Commit(0, false)
if err != nil {
return nil, err
}
// 4. 제네시스 블록 헤더 생성
head := &types.Header{
Number: new(big.Int).SetUint64(g.Number),
Nonce: types.EncodeNonce(g.Nonce),
Time: g.Timestamp,
ParentHash: g.ParentHash,
```

```
Extra: g.ExtraData,

GasLimit: g.GasLimit,

GasUsed: g.GasUsed,

Difficulty: g.Difficulty,

MixDigest: g.Mixhash,

Coinbase: g.Coinbase,

Root: root,

}

// 5. 제네시스 블록 생성 및 저장

block := types.NewBlock(head, nil, nil, nil, trie.NewStackTrie(nil))

rawdb.WriteBlock(db, block)

rawdb.WriteReceipts(db, block.Hash(), block.NumberU64(), nil)

rawdb.WriteCanonicalHash(db, block.Hash(), block.NumberU64())

rawdb.WriteHeadBlockHash(db, block.Hash())

rawdb.WriteHeadHeaderHash(db, block.Hash())

return  block, nil

}
```

🔧 주요 구성 요소

제네시스 설정 구조

| 필드 | 설명 | 예시 |

|------|------|------|

| config | 체인 설정 (하드포크 블록 번호 등) | {"chainId": 1337} |

| difficulty | 초기 채굴 난이도 | "0x1" |

| gasLimit | 블록 가스 한도 | "0x8000000" |

| alloc | 초기 계정 할당 | 계정별 잔액, 코드, 스토리지 |

| timestamp | 제네시스 블록 타임스탬프 | "0x0" |

| extraData | 추가 데이터 | "0x00" |

데이터베이스 구조

체인 데이터베이스 키-값 저장:

```
// 주요 저장 키들
var (
headBlockKey = []byte("LastBlock")
headHeaderKey = []byte("LastHeader")
headFastBlockKey = []byte("LastFast")
// 블록 관련
headerPrefix = []byte("h") // headerPrefix + num (uint64 big endian) + hash -> h
blockBodyPrefix = []byte("b") // blockBodyPrefix + num (uint64 big endian) + has
blockReceiptsPrefix = []byte("r") // blockReceiptsPrefix + num (uint64 big endia
// 설정
configPrefix = []byte("ethereum-config-") // config prefix for the db
)
```

📊 실행 예시 및 로그

성공적인 초기화 로그:

```
$ geth init genesis.json
INFO [날짜] Successfully wrote genesis state database=chaindata hash=0x1234...ab
```

상세 로그 (-verbosity 5):

```
$ geth --verbosity 5 init genesis.json
DEBUG[날짜] Reading genesis file path=genesis.json
DEBUG[날짜] Parsed genesis config chainId=1337 difficulty=1
DEBUG[날짜] Creating chain database path=/home/user/.ethereum/geth/chaindata
DEBUG[날짜] Initializing trie database preimages=false verkle=false
DEBUG[날짜] Committing genesis state accounts=1 root=0x5678...efab
INFO [날짜] Successfully wrote genesis state database=chaindata hash=0x1234...ab
```

🛠️ 문제 해결

일반적인 오류들

1. 제네시스 파일 형식 오류:

```
Fatal: invalid genesis file: invalid character '}' looking for beginnin
```

해결: JSON 문법 검증 및 수정

2. 데이터 디렉토리 권한 오류:

```
Fatal: Failed to create the protocol stack: mkdir /root/.ethereum: perm
```

해결: `--datadir` 플래그로 쓰기 가능한 디렉토리 지정

3. 기존 체인과 호환성 오류:

```
Fatal: Failed to write genesis block: genesis block mismatch
```

해결: 기존 데이터 삭제 후 재초기화 또는 다른 데이터 디렉토리 사용

고급 사용법

1. 커스텀 데이터 디렉토리:

```
geth --datadir ./custom-chain init genesis.json
```

2. 프리이미지 캐시 활성화:

```
geth --cache.preimages init genesis.json
```

3. 하드포크 오버라이드:

```
geth --override.osaka 1000000 init genesis.json
```

🔍 내부 데이터 구조

생성되는 주요 파일들:

```
<datadir>/geth/
├── chaindata/ # 체인 데이터베이스
| ├── 000001.log # LevelDB 로그 파일
| ├── CURRENT # 현재 매니페스트 파일
| ├── LOCK # 데이터베이스 락 파일
| └── MANIFEST-000000 # 매니페스트 파일
├── lightchaindata/ # 라이트 클라이언트 데이터
└── nodes/ # 노드 키 저장
```

메모리 사용량:

- **최소 메모리**: ~50MB (기본 설정)
- **권장 메모리**: ~100MB (프리이미지 캐시 포함)
- **디스크 사용량**: ~10MB (빈 체인 기준)

📈 성능 최적화

빠른 초기화를 위한 팁:

1. **SSD 사용**: 디스크 I/O 성능 향상
2. **충분한 메모리**: 트라이 캐시 효율성 증대
3. **단순한 제네시스**: 불필요한 초기 계정 최소화

참고 자료:

- Genesis 파일 형식
- 체인 설정 옵션
- 데이터베이스 구조

이 문서는 go-ethereum 소스 코드 분석을 바탕으로 작성되었습니다.